DATA SCIENCE MINOR PROJECT REPORT

(Project Semester August–December 2026)

SEOUL BIKE SHARING DEMAND ANALYSIS AND PREDICTIONS

Submitted by

Mohit

Registration No:12418504

Program and Section: B.Tech Computer Science and Engineering

Course Code: **INT234**

Under the Guidance of instructor

Ms. Maneet Kaur

Discipline of CSE

Lovely School of Computer Science and Engineering

Lovely Professional University, Phagwara



Certificate

This is to certify that **Mohit**, bearing Registration No. **12418504,** has completed the INT234 project titled "**Seoul Bike Sharing Demand Analysis and Prediction**" under my guidance and supervision. To the best of my knowledge, the present work is the result of his/her original development, effort and study.

Signature and Name of the Supervisor: Ms. Maneet Kaur
Designation of the Supervisor: Faculty
Lovely School of Computer Science and Engineering
Lovely Professional University Phagwara,
Punjab.

Date: 22/10/26

Declaration

I, **Mohit**, student of B.Tech (Computer Science and Engineering) under the CSE/IT Discipline at Lovely Professional University, Punjab, hereby declare that all the information furnished in this project report is based on my own intensive work and is genuine.

Date: **22** / **10** / **2026**

Signature: Mohit

Registration No: **12418504**

Name of the Student: Mohit

Acknowledgement

I would like to thank my faculty guide, Ms. Maneet Kaur, for the direction and feedback I received while working on this project. The clear guidelines about what a good data science report should contain helped me plan the work and correct several mistakes in my early drafts.

I am grateful to Lovely Professional University and the Discipline of CSE for giving me the chance to work on a real dataset instead of only textbook exercises. I also thank the creators of the Seoul Bike Sharing Demand dataset and the open-source community behind Python, pandas, NumPy, Matplotlib, Seaborn, SciPy, scikit-learn and mlxtend. Finally, I thank my family, friends and classmates for their support.

Mohit

Table of Contents

Certificate 2

Declaration 3

Acknowledgement 4

1. Introduction 7

1.1 Background and Motivation 7

1.2 Problem Statement 7

1.3 Objectives of the Project 7

1.4 Tools and Technologies 8

1.5 Methodology Overview 8

1.6 Organisation of the Report 8

2. Source of Dataset 9

2.1 About the Dataset 9

2.2 Variables Used 9

2.3 Suitability and Limitations of the Data 9

3. EDA Process 10

3.1 Loading the Data 10

3.2 First Look at the Data 10

3.3 Data Cleaning 10

3.4 Feature Engineering 10

3.5 Preparation for Machine Learning 10

4. Analysis on the Dataset 12

4.1 Distribution of Hourly Demand (Objective 1) 12

4.2 Demand by Hour, Season and Month (Objective 2) 12

4.3 Weather and Demand (Objective 3) 13

4.4 Regression Models (Objective 4) 15

4.5 Comparison of Regression Models and Best-Fit Selection (Objective 4) 17

4.6 Logistic Regression: Demand Level Classification (Objective 5a) 18

4.7 Decision Tree Classification (Objective 5b) 19

4.8 Support Vector Machine Classification (Objective 5c) 19

4.9 Comparison of Classifiers and Best-Fit Selection 20

4.10 K-Means Clustering (Objective 6) 20

4.11 Hierarchical Clustering (Objective 7) 21

4.12 Association Rule Mining (Objective 8) 22
5. Conclusion 24
6. Future Scope 26
7. References 27
Appendix A: GitHub and LinkedIn Screenshots 28
A.1 GitHub Repository 28
A.2 LinkedIn Post 28

1. Introduction

1.1 Background and Motivation

Bike-sharing systems are an important part of modern urban transport. They reduce traffic and pollution, but they only work well when the right number of bikes is available at the right time. Demand is not constant: it depends on the hour of the day, the season, the weather and whether the day is a holiday. If an operator can estimate demand in advance, it can plan bike redistribution, maintenance and staffing more effectively.

This project studies the Seoul Bike Sharing Demand dataset, which records the number of bikes rented every hour in Seoul together with weather and calendar information. I wanted a problem that uses every major technique of the INT234 course, namely regression, classification, clustering and association rule mining, on one clean real-world dataset.

1.2 Problem Statement

How does hourly bike rental demand in Seoul depend on weather conditions and time-related factors, and how well can this demand be (a) predicted as a number, (b) classified into Low, Medium and High levels, (c) grouped into meaningful usage patterns, and (d) described using association rules?

1.3 Objectives of the Project

Eight objectives were defined. They are listed in Table 1 with the technique used for each.

Table 1: Project objectives and the technique used

No.	Objective	Technique / Visual
1	Analyse the distribution of hourly bike rental demand	Histogram, descriptive statistics
2	Study demand patterns by hour, season and month	Line plot, bar plots, heatmap
3	Examine the relationship between weather and demand	Correlation heatmap, scatter plots, box plot
4	Predict hourly demand and select the best-fit regression model	Simple, Multiple and Polynomial Regression against a baseline
5	Classify hourly demand as Low / Medium / High	Logistic Regression, Decision Tree, SVM
6	Group similar hours using K-Means	K-Means, elbow method, silhouette score, PCA

No.	Objective	Technique / Visual
7	Compare hierarchical clustering linkages	Agglomerative clustering, dendrogram
8	Discover relations between time, weather and demand	Apriori, association rules (support, confidence, lift)

Objectives 4 to 6 together use more than four machine learning models (three regression models, three classifiers and two clustering families), which satisfies the requirement of implementing at least four models and choosing a best-fit model.

1.4 Tools and Technologies

Table 2: Software and libraries used

Tool / Library	Purpose in this project
Python (Spyder IDE)	Writing and running the complete script
pandas, NumPy	Loading, cleaning, grouping, feature engineering
Matplotlib, Seaborn	All charts and heatmaps
SciPy	Dendrogram for hierarchical clustering
scikit-learn	Pipelines, scaling, models, evaluation metrics, PCA
mlxtend	Apriori algorithm and association rules
GitHub / LinkedIn	Storing the code and sharing the results

1.5 Methodology Overview

1. Load the CSV file and inspect shape, data types, missing values and duplicates.
2. Keep only the observations where the bike system was functioning.
3. Engineer features: month, day of week and cyclical (sine/cosine) encodings of hour, month and day.
4. Explore the data with graphs that answer Objectives 1 to 3.
5. Split the data chronologically into 80% training and 20% testing.
6. Train and evaluate regression models, classifiers, clustering models and association rules.
7. Compare models, select the best-fit ones, and summarise the findings.

1.6 Organisation of the Report

Section 2 describes the dataset. Section 3 explains the cleaning and feature engineering. Section 4 is the main analysis; each analysis follows five labels: introduction, general description, specific requirements/functions/formulas, analysis results and visualization. Sections 5 to 7 give the conclusion, future scope and references. The appendix contains the GitHub and LinkedIn screenshots.

2. Source of Dataset

2.1 About the Dataset

The project uses the **Seoul Bike Sharing Demand** dataset (SeoulBikeData.csv) from the UCI Machine Learning Repository / Kaggle. It contains **8,760 hourly observations** (24 hours × 365 days) from **1 December 2017 to 30 November 2018** and **14 attributes**. There are no missing values and no duplicate rows. The target variable is **Rented Bike Count**.

2.2 Variables Used

Table 3: Description of the dataset columns

Column	Meaning	Type
Date	Date of observation (dd/mm/yyyy)	Date
Rented Bike Count	Bikes rented in that hour (target)	Integer
Hour	Hour of the day (0–23)	Integer
Temperature(°C)	Air temperature	Float
Humidity(%)	Relative humidity	Integer
Wind speed (m/s)	Wind speed	Float
Visibility (10m)	Visibility in units of 10 m	Integer
Dew point temperature(°C)	Dew point	Float
Solar Radiation (MJ/m2)	Solar radiation	Float
Rainfall(mm)	Rainfall	Float
Snowfall (cm)	Snowfall	Float
Seasons	Winter, Spring, Summer, Autumn	Category
Holiday	Holiday / No Holiday	Category
Functioning Day	Whether the system was operating (Yes/No)	Category

2.3 Suitability and Limitations of the Data

The dataset suits the problem because it combines time and weather variables with a clear numeric target, is complete, and has enough rows for modelling. Its limitations are:

- It covers only one year and one city, so yearly trends and other cities cannot be studied.
- It has no information on bike stations, trip locations, events or user types.
- Rows are hourly observations in time order, so neighbouring hours are related. A random split would leak information; I therefore used a chronological split.

3. EDA Process

3.1 Loading the Data

The file is read with ISO-8859-1 encoding because column names contain the degree symbol. Column names are stripped of spaces and the Date column is converted to datetime.

```
df = pd.read_csv(DATA_PATH, encoding="ISO-8859-1")
df.columns = df.columns.str.strip()
df["Date"] = pd.to_datetime(df["Date"], format="%d/%m/%Y")
```

3.2 First Look at the Data

Output of the script:

```
Shape: (8760, 14)   Missing Values: 0   Duplicates: 0
Rented Bike Count: mean 704.60, median 504.50, std 645.00, min 0, max 3556
```

The mean is larger than the median, which already suggests a right-skewed target.

3.3 Data Cleaning

- **Duplicates:** none were found, but `drop_duplicates()` is kept as a safety step.
- **Non-functioning days:** 295 rows have Functioning Day = No, and all have zero rentals because the system was closed. They are not a demand pattern, so they were removed. The modelling dataset has **8,465 rows**.
- **Dew point temperature:** it is highly correlated with temperature (about 0.91) and was left out of the model features to avoid multicollinearity.

3.4 Feature Engineering

Month and day of week were extracted from the date. Hour, month and day of week are cyclical (23:00 is next to 00:00), so each was encoded with sine and cosine:

$$\text{Hour_sin} = \sin(2\pi \times \text{Hour} / 24) \qquad \text{Hour_cos} = \cos(2\pi \times \text{Hour} / 24)$$

```
for name, col, period, shift in [("Hour","Hour",24,0), ("Month","Month",12,1),
("Day","DayOfWeek",7,0)]:
    angle = 2*np.pi*(m[col]-shift)/period
    m[name+"_sin"], m[name+"_cos"] = np.sin(angle), np.cos(angle)
```

3.5 Preparation for Machine Learning

1. **Chronological split:** the first 80% of the dates (1 Dec 2017 to 10 Sep 2018) form the training set (**6,768 rows**) and the remaining dates (11 Sep to 30 Nov 2018) form the test set (**1,697 rows**). Whole days stay together.
2. **Scaling and encoding:** numeric features are standardised, and Seasons and Holiday are one-hot encoded inside a scikit-learn `ColumnTransformer` / `Pipeline`, so that the test data never influences the preprocessing.

3. **Features used (15):** temperature, humidity, wind speed, visibility, solar radiation, rainfall, snowfall, six cyclical time features, Seasons and Holiday.

4. Analysis on the Dataset

4.1 Distribution of Hourly Demand (Objective 1)

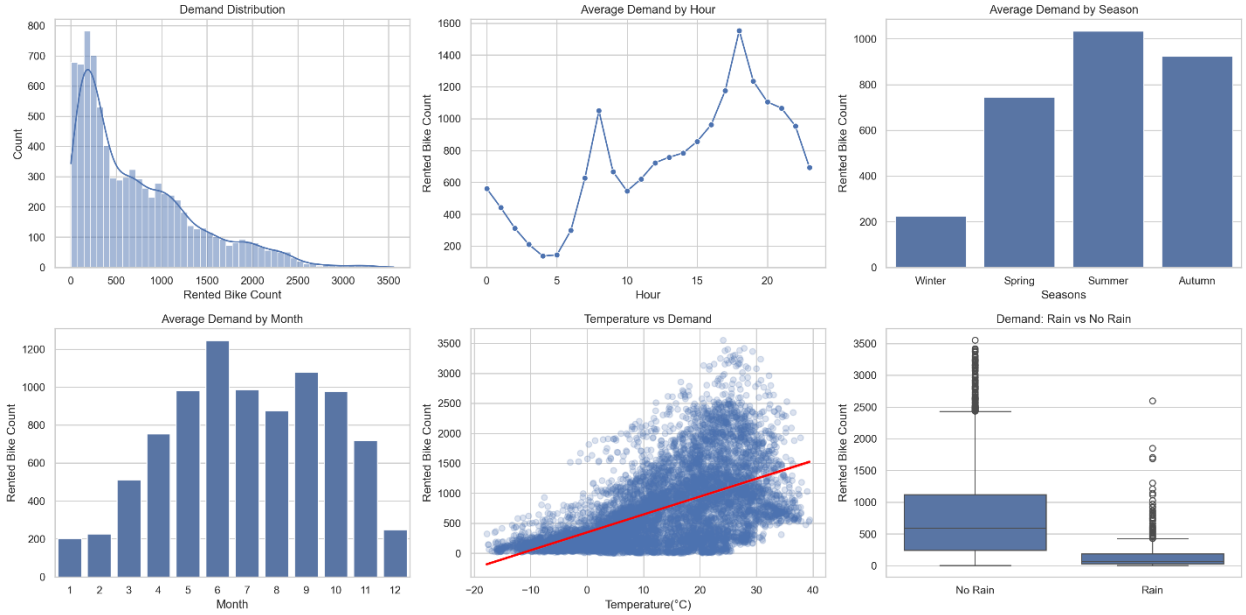
(i) Introduction. Before modelling, I wanted to know how demand is spread across hours.

(ii) General Description. A histogram with a density curve counts how many hours fall into each demand range.

(iii) Specific Requirements, Functions and Formulas. `sns.histplot(m[T], bins=50, kde=True)`. Mean = $(1/n)\ \Sigma\ y_i$.

(iv) Analysis Results. The average demand is 704.6 bikes per hour with a standard deviation of 645.0. The median (504.5) is below the mean and the maximum is 3,556, so the distribution is right-skewed: most hours have low or moderate demand and a few peak hours have very high demand.

(v) Visualization.



4.2 Demand by Hour, Season and Month (Objective 2)

(i) Introduction. Bike use follows daily routines and the calendar, so I compared demand across hours, seasons and months.

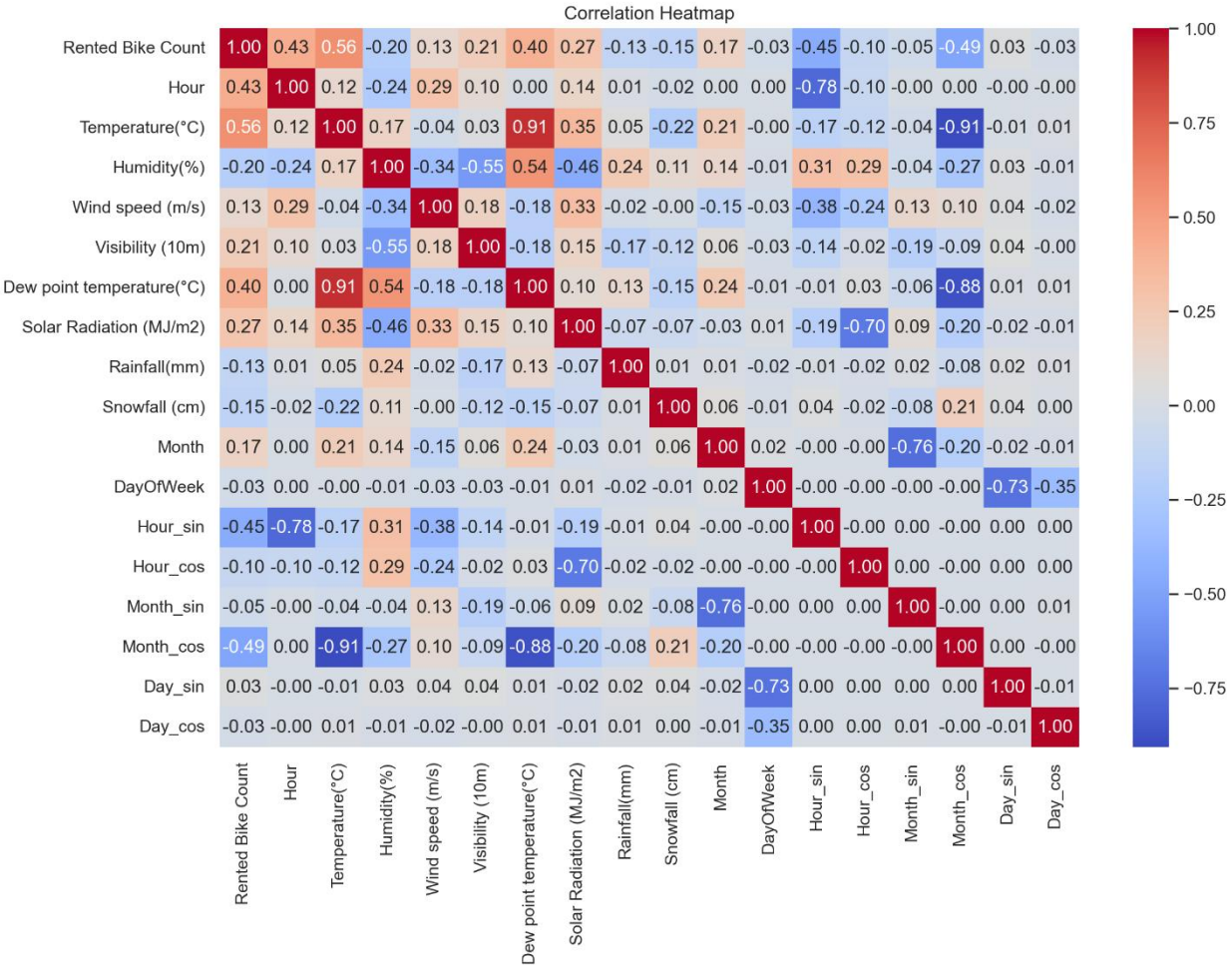
(ii) General Description. Average demand was calculated for each group and shown as a line plot (hour), bar plots (season, month) and a heatmap (hour × season).

(iii) Specific Requirements, Functions and Formulas. `sns.lineplot`, `sns.barplot`, `pivot_table(index="Hour", columns="Seasons", values=T)`.

(iv) Analysis Results.

- **Hourly:** demand has two peaks: 08:00 (about 1,016 rentals) and a stronger evening peak at 18:00 (about 1,503). The lowest demand is at 04:00 (about 133). This matches commuting hours.
- **Seasonal:** Summer 1,034, Autumn 820, Spring 730 and Winter 226 average rentals per hour. Winter demand is only about one fifth of summer demand.
- **Monthly:** June has the highest and December the lowest average demand.
- **Holiday:** about 715 rentals on non-holidays against about 500 on holidays, which suggests many trips are work or study commutes.

(v) Visualization.



4.3 Weather and Demand (Objective 3)

(i) Introduction. Weather is expected to strongly influence cycling.

(ii) General Description. A correlation matrix, a temperature regression scatter plot and a rain/no-rain box plot show how each weather variable relates to demand.

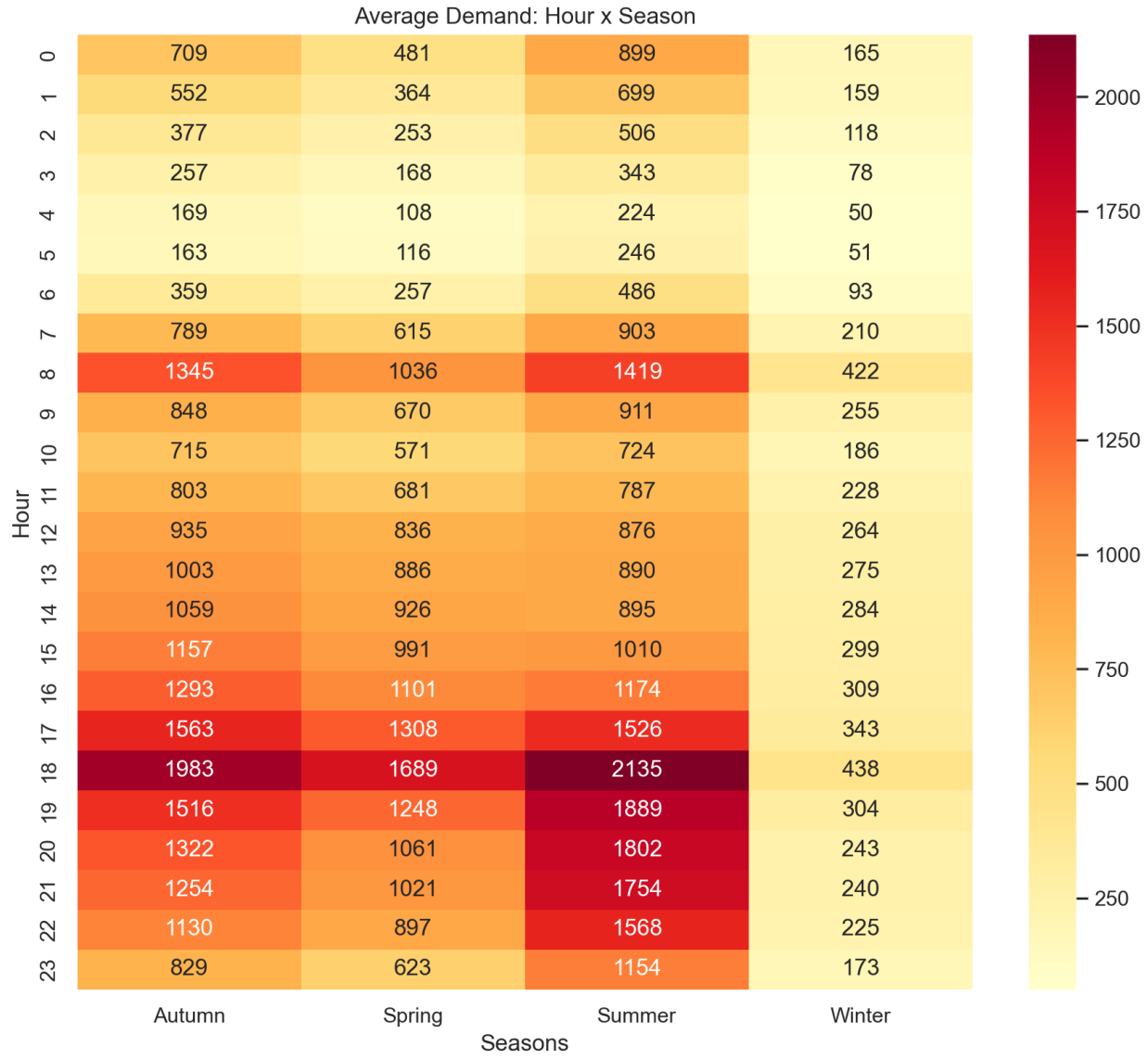
(iii) Specific Requirements, Functions and Formulas. Pearson correlation: $r = \Sigma(x-\bar{x})(y-\bar{y}) / \sqrt{(\Sigma(x-\bar{x})^2\ \Sigma(y-\bar{y})^2)}$, computed with `m.corr()`.

(iv) Analysis Results.

Feature	Correlation with demand
Temperature	0.563
Hour	0.425
Dew point temperature	0.400
Solar radiation	0.274
Visibility	0.212
Humidity	−0.202
Snowfall	−0.152
Rainfall	−0.129

Temperature is the strongest single numeric driver. Humidity, snowfall and rainfall reduce demand. All correlations are moderate at best, which already tells us that no single variable explains demand; this is why the regression models in Section 4.4 reach only a moderate R^2. Correlation does not prove causation.

(v) Visualization.



4.4 Regression Models (Objective 4)

(i) Introduction. Here I predict the exact number of bikes rented in an hour.

(ii) General Description. Four models were trained on the same training data: a **Baseline** that always predicts the training mean (to show what "no skill" looks like), **Simple Linear Regression** (temperature only), **Multiple Linear Regression** (all 15 features) and **Polynomial Regression** (degree 2 on the numeric features, which adds squared and interaction terms).

(iii) Specific Requirements, Functions and Formulas.

$$\hat{y} = \beta 0 + \beta 1x1 + \beta 2x2 + \ldots + \beta pxp$$

$$MAE = (1/n)\,\Sigma|y_i - \hat{y}_i| \qquad MSE = (1/n)\,\Sigma(y_i - \hat{y}_i)^2$$

$$RMSE = \sqrt{MSE} \qquad R^2 = 1 - \Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y})^2$$

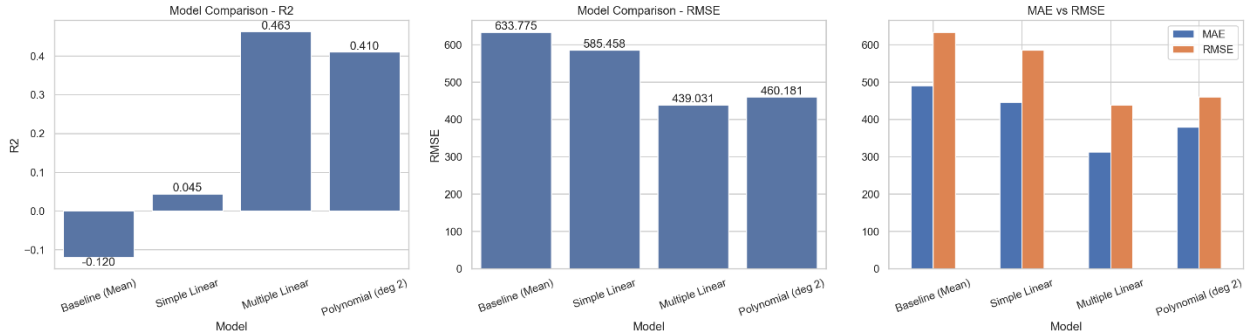
```
Pipeline([("prep", poly_prep), ("model", LinearRegression())])
```

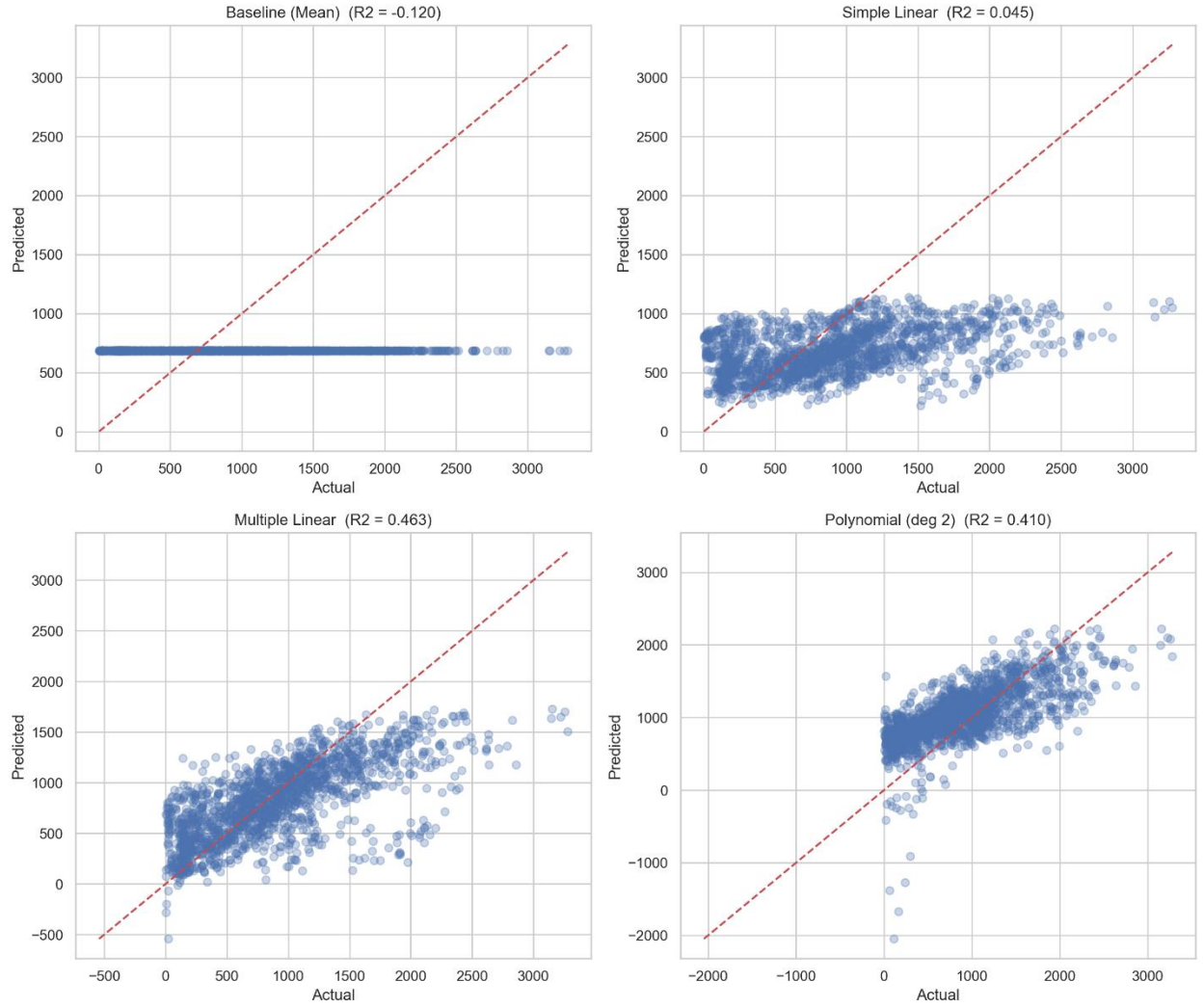
(iv) Analysis Results. The script printed:

Model	MAE	MSE	RMSE	R^2
Simple Linear Regression	446.486	342,761.304	585.458	0.045
Multiple Linear Regression	313.109	192,748.541	439.031	**0.463**
Polynomial Regression	379.175	211,766.253	460.181	0.410
Baseline (mean)	490.880	401,670.537	633.775	−0.120

- Temperature alone explains only 4.5% of the variation on the test set, because demand also depends strongly on the hour of the day.
- Adding time and weather features raises R^2 from 0.045 to 0.463, and cuts MAE from 446 to 313 bikes.
- Polynomial Regression, which has over a hundred terms, performed worse than the plain linear model (R^2 0.410), which suggests overfitting to the training period.
- The baseline has a negative R^2 because the test period has a higher average demand (894.8) than the training period (687.6). This **distribution shift** (the test period is Sep–Nov, after the summer peak in the training data) makes every model harder to evaluate and is the main reason R^2 stays below 0.5.

(v) Visualization.





4.5 Comparison of Regression Models and Best-Fit Selection (Objective 4)

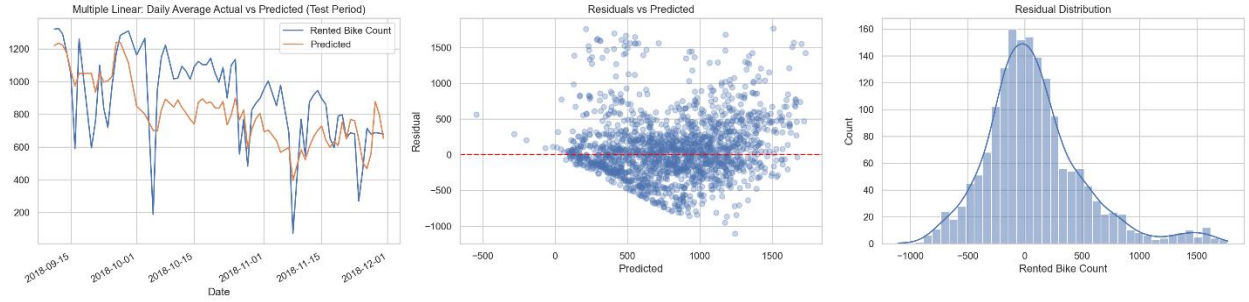
(i) Introduction. The models must be compared fairly to choose one.

(ii) General Description. The metrics of all models are placed in one table and compared in bar charts. The model with the highest R^2 (and lowest RMSE) on unseen test data is the best fit.

(iii) Specific Requirements, Functions and Formulas. `best = reg.loc[reg["R2"].idxmax(), "Model"]`

(iv) Analysis Results. Multiple Linear Regression is the best-fit regression model (R^2 = 0.4627, RMSE = 439.03, MAE = 313.11). It is also the simplest model that uses all the features, so it is easy to explain. An R^2 of 0.46 is moderate: roughly half the variation in demand is still unexplained, since the dataset has no information on events, station locations or user behaviour.

(v) Visualization.



4.6 Logistic Regression: Demand Level Classification (Objective 5a)

(i) Introduction. For planning, an operator often needs "will demand be low, medium or high?" rather than an exact number.

(ii) General Description. Demand was converted into three classes using the 33rd and 67th percentiles of the **training** data only (Low ≤ 254, Medium 255–803, High > 803), so no test information leaks into the labels. Logistic Regression estimates the probability of each class and chooses the highest, using `class_weight="balanced"`.

(iii) Specific Requirements, Functions and Formulas.

$$P(\text{class } k \mid x) = e^{z_k} / \Sigma_j e^{z_j}, \quad z_k = w_k \cdot x + b_k$$
$$\text{Accuracy} = (TP+TN)/\text{Total} \quad \text{Precision} = TP/(TP+FP) \quad \text{Recall} = TP/(TP+FN)$$
$$F1 = 2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision}+\text{Recall})$$

(iv) Analysis Results. Accuracy 0.626, macro precision 0.608, macro recall 0.625, macro F1 0.606, log loss 0.815, ROC AUC 0.808. By class: High (precision 0.88, recall 0.66), Low (0.50, 0.67), Medium (0.45, 0.55). The model is very reliable when it says "High" but often confuses Low and Medium. Medium is the hardest class because it sits between the two thresholds.

(v) Visualization.



4.7 Decision Tree Classification (Objective 5b)

(i) Introduction. A decision tree can capture non-linear rules such as "warm and evening means high demand".

(ii) General Description. The tree asks a series of yes/no questions about the features and ends at a class. It was limited to max_depth=12 and min_samples_leaf=10 to reduce overfitting.

(iii) Specific Requirements, Functions and Formulas. Gini impurity $G = 1 - \Sigma\, p_k^2$; the split with the largest impurity reduction is chosen. DecisionTreeClassifier(max_depth=12, min_samples_leaf=10, class_weight="balanced", random_state=42).

(iv) Analysis Results. Accuracy 0.702, macro precision 0.699, macro recall 0.715, macro F1 **0.694**, log loss 3.608, ROC AUC 0.851. By class: High (precision 0.93, recall 0.68), Low (0.64, 0.76), Medium (0.52, 0.71). The tree has the best macro F1 and finds Low-demand hours best, but its log loss is very high: a tree gives hard 0/1-like probabilities, so its wrong predictions are heavily penalised. Its probabilities are therefore not reliable.

09_Classification_Comparison.png (right panel, feature importances)]

4.8 Support Vector Machine Classification (Objective 5c)

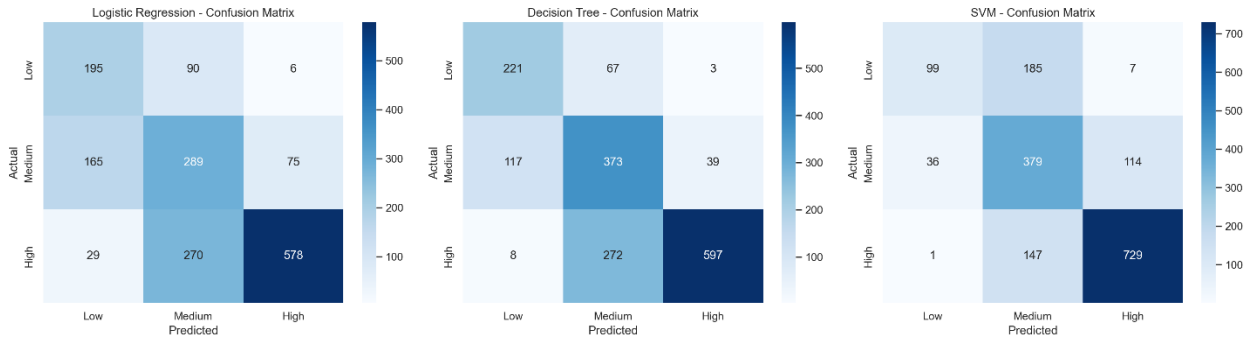
(i) Introduction. SVM with a non-linear kernel is a strong general-purpose classifier for mixed features.

(ii) General Description. The SVM finds the boundary with the widest margin between classes. The RBF kernel maps the data to a higher-dimensional space so curved boundaries become possible.

(iii) Specific Requirements, Functions and Formulas. $K(x, x') = \exp(-\gamma \|x - x'\|^2)$; SVC(kernel="rbf", C=1.0, gamma="scale", class_weight="balanced", probability=True).

(iv) Analysis Results. Accuracy **0.711**, macro precision 0.706, macro recall 0.629, macro F1 0.640, log loss **0.636**, ROC AUC **0.876**. By class: High (precision 0.86, recall 0.83), Low (0.73, **0.34**), Medium (0.53, 0.72). SVM is the best at ranking and probability quality, but it misses many Low-demand hours (recall only 0.34), which pulls its macro F1 below that of the Decision Tree.

(v) Visualization.



4.9 Comparison of Classifiers and Best-Fit Selection

(i) Introduction. The three classifiers are compared on the same test set.

(ii) General Description. Accuracy, macro precision, macro recall, macro F1, log loss and ROC AUC are compared in one table and one bar chart.

(iii) Specific Requirements, Functions and Formulas. `roc_auc_score(..., multi_class="ovr")`, `log_loss`, `classification_report`.

(iv) Analysis Results.

Model	Accuracy	Precision	Recall	Macro F1	Log Loss	ROC AUC
Logistic Regression	0.626	0.608	0.625	0.606	0.815	0.808
Decision Tree	0.702	0.699	0.715	**0.694**	3.608	0.851
SVM	**0.711**	0.706	0.629	0.640	**0.636**	**0.876**

SVM is selected as the best overall classifier because it has the highest accuracy and ROC AUC and the lowest log loss. The Decision Tree is the best choice if the priority is balanced detection of all three classes (highest macro F1). Note that the test set has 877 High, 529 Medium and 291 Low hours, while the training set was almost perfectly balanced (about 2,240 hours per class). The test period is therefore much more demand-heavy, which makes the Low class small and hard to recall.

4.10 K-Means Clustering (Objective 6)

(i) Introduction. Without using labels, can hours be grouped by similar weather, time and demand profile?

(ii) General Description. K-Means assigns each observation to the nearest of k centres and moves the centres to the mean of their members until the groups stop changing. Demand was included as a profile feature, so the clusters are descriptive rather than predictive.

(iii) Specific Requirements, Functions and Formulas. Features: temperature, humidity, wind speed, solar radiation, Hour_sin, Hour_cos, demand and one-hot Seasons, all standardised. Objective: $J = \Sigma_k \Sigma_{(x \in C_k)} \|x - \mu_k\|^2$. The elbow method (inertia) and the silhouette score $s = (b - a)/\max(a, b)$ were calculated for k = 2 to 8. `KMeans(n_clusters=3, random_state=42, n_init=10)`.

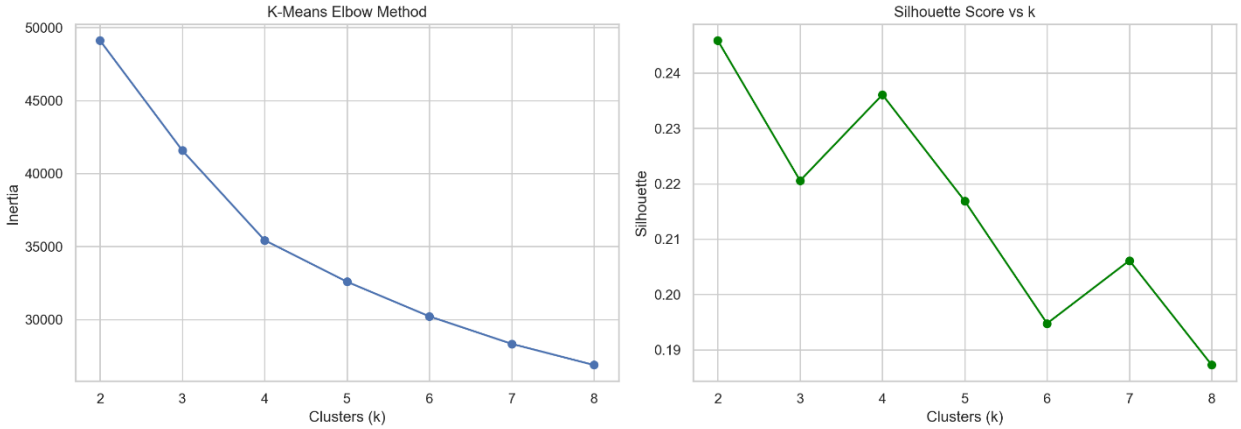
(iv) Analysis Results. With k = 3 the silhouette score is **0.2205**, which indicates overlapping but interpretable clusters. The cluster profiles are:

Cluster	Average temperature (°C)	Average demand	Interpretation
0	14.51	552	Mild-weather, moderate-demand hours

Cluster	Average temperature (°C)	Average demand	Interpretation
			(spring/autumn)
1	−1.63	295	Cold-weather, low-demand hours (winter)
2	22.53	1,351	Warm-weather, high-demand hours (summer, daytime and evening)

The clusters therefore follow a temperature–season–demand gradient, which agrees with the correlation and seasonal findings.

(v) Visualization.



4.11 Hierarchical Clustering (Objective 7)

(i) Introduction. Hierarchical clustering gives a second opinion on the cluster structure and does not need centres to be chosen in advance.

(ii) General Description. Agglomerative clustering starts with every observation as its own cluster and repeatedly merges the two closest clusters. The linkage rule defines "closest": Ward (minimum increase in variance), complete (maximum distance) and average (mean distance).

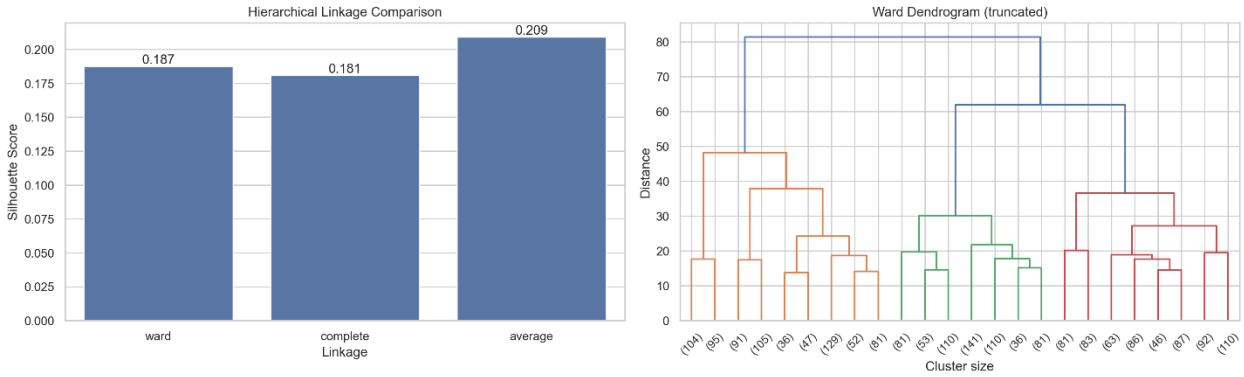
(iii) Specific Requirements, Functions and Formulas. Because the algorithm needs a full distance matrix, a reproducible random sample of 2,000 rows was used (`random_state=42`). `AgglomerativeClustering(n_clusters=3, linkage=...)`, `scipy.cluster.hierarchy.dendrogram`.

(iv) Analysis Results.

Linkage	Silhouette Score
Ward	0.1875
Complete	0.1808
Average	**0.2091**

Average linkage scored highest, but all values are close to the K-Means score (0.2205), so K-Means with k = 3 is the better and cheaper clustering for this data. The low scores show that demand patterns change gradually and the groups overlap.

(v) Visualization



4.12 Association Rule Mining (Objective 8)

(i) Introduction. Association rules express findings as readable "if … then …" statements, such as "if summer and afternoon, then high demand".

(ii) General Description. Each hour is treated as a basket of items: season, holiday status, hour group (Night, Morning, Afternoon, Evening), temperature group (Cold, Mild, Warm, Hot), humidity group, precipitation and demand level (using the same training-derived thresholds). The Apriori algorithm finds item combinations that occur often.

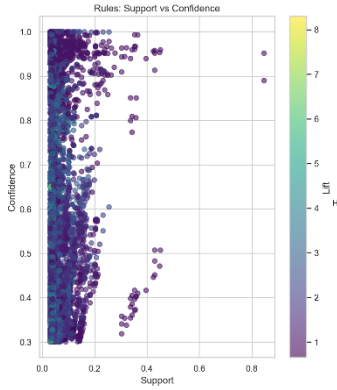
(iii) Specific Requirements, Functions and Formulas.

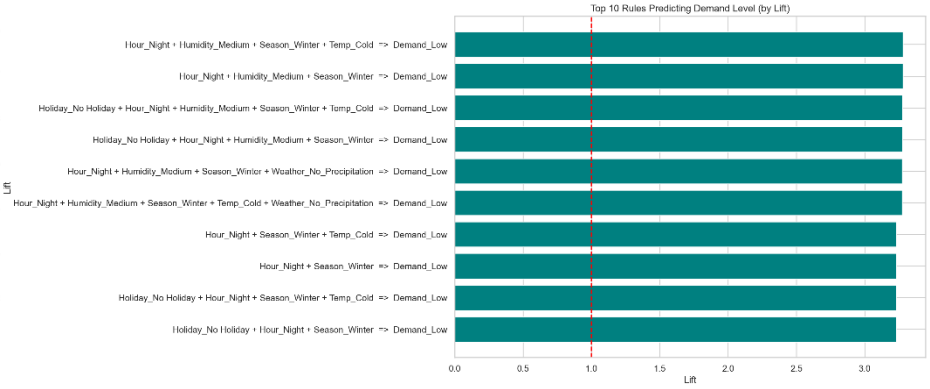
Support(A) = count(A) / N
Confidence(A → B) = Support(A ∪ B) / Support(A)
Lift(A → B) = Confidence(A → B) / Support(B)
apriori(basket, min_support=0.03) and association_rules(freq, metric="confidence", min_threshold=0.30). Lift above 1 means A and B occur together more than expected by chance.

(iv) Analysis Results. Single-item supports from the script output: Demand High 0.367, Medium 0.334, Low 0.299; Cold temperature 0.422 (the most common temperature group); Hot 0.060; Holiday 0.048; Evening 0.292; Summer 0.261. Rules with the highest lift involve hot, summer-afternoon and no-precipitation conditions together, which agrees with the high-demand cluster found earlier.

[Paste here the top 5–10 rules predicting Demand_High and Demand_Low (antecedent, support, confidence, lift) from the console output or Results/association_rules.csv.]

(v) Visualization.





5. Conclusion

This project analysed 8,465 functioning hourly observations of the Seoul bike-sharing system (Dec 2017 – Nov 2018). The main findings are:

- Demand is right-skewed (mean 704.6, median 504.5, maximum 3,556) and follows commuting patterns, with peaks at 08:00 and 18:00 and a minimum at about 04:00.
- Season and weather matter strongly: summer demand (about 1,034) is more than four times winter demand (about 226), and temperature is the strongest single driver ($r = 0.563$). Rain, snow and humidity reduce demand.
- **Multiple Linear Regression was the best regression model** (R^2 0.463, RMSE 439), clearly better than temperature alone (R^2 0.045) and than the baseline. Polynomial Regression (R^2 0.410) overfitted.
- **SVM was the best classifier** (accuracy 0.711, ROC AUC 0.876, log loss 0.636), while the Decision Tree gave the best macro F1 (0.694). Medium demand was the hardest level to identify.
- K-Means (silhouette 0.2205) found three interpretable groups: cold/low-demand, mild/moderate-demand and warm/high-demand hours. Hierarchical clustering (best silhouette 0.2091, average linkage) gave a similar but slightly weaker structure.
- Association rules link hot, summer, afternoon, dry conditions with high demand.

Table 4: Summary of the models

Model	Task	Main metric	Remark
Simple Linear Regression	Predict demand	R^2 0.045	Temperature alone is insufficient
Multiple Linear Regression	Predict demand	R^2 0.463	Best-fit regression model
Polynomial Regression	Predict demand	R^2 0.410	Overfitting
Logistic Regression	Classify demand level	Accuracy 0.626	Weakest classifier
Decision Tree	Classify demand level	Macro F1 0.694	Best balance across classes
SVM	Classify demand level	Accuracy 0.711, AUC 0.876	Best overall classifier
K-Means ($k = 3$)	Cluster hours	Silhouette 0.2205	Best clustering
Hierarchical (average)	Cluster hours	Silhouette 0.2091	Similar structure

Practical recommendations. Operators should keep the largest fleet available at the 08:00 and 18:00 peaks and on warm, dry summer days, schedule maintenance and rebalancing during cold winter periods and night hours, and use weather forecasts as an early signal for demand.

Limitations. The results show associations, not cause and effect. The data covers a single year, and the test period (Sep–Nov) has a different demand level from the training period, so scores are lower than a random split would show.

6. Future Scope

- **Better regression models:** Random Forest, Gradient Boosting or XGBoost, and regularised models (Ridge, Lasso) to control the overfitting of polynomial features.
- **Time-series features:** lagged demand (demand one hour or one day earlier) and time-series cross-validation instead of a single split.
- **More data:** several years of data would remove the distribution-shift problem and allow trend analysis; public holidays, events and station-level data would add useful information.
- **Hyperparameter tuning:** GridSearchCV for SVM and Decision Tree settings, and probability calibration for the Decision Tree.
- **Explainability:** permutation importance or SHAP values.
- **Deployment:** a Streamlit dashboard where an operator selects the weather and hour and receives the predicted demand level.

7. References

[1] Seoul Bike Sharing Demand dataset. UCI Machine Learning Repository / Kaggle. Original data: Seoul Metropolitan Government and Korea Meteorological Administration.

[2] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

[3] McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference, 56–61.

[4] Harris, C. R. et al. (2020). Array programming with NumPy. Nature, 585, 357–362.

[5] Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3), 90–95.

[6] Raschka, S. (2018). MLxtend: Providing machine learning and data science utilities and extensions to Python's scientific computing stack. Journal of Open Source Software, 3(24), 638.

[7] Agrawal, R. and Srikant, R. (1994). Fast Algorithms for Mining Association Rules. Proceedings of the 20th VLDB Conference, 487–499.

[8] MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. Proceedings of the Fifth Berkeley Symposium, 1, 281–297.

[9] James, G., Witten, D., Hastie, T. and Tibshirani, R. (2021). An Introduction to Statistical Learning (2nd ed.). Springer.

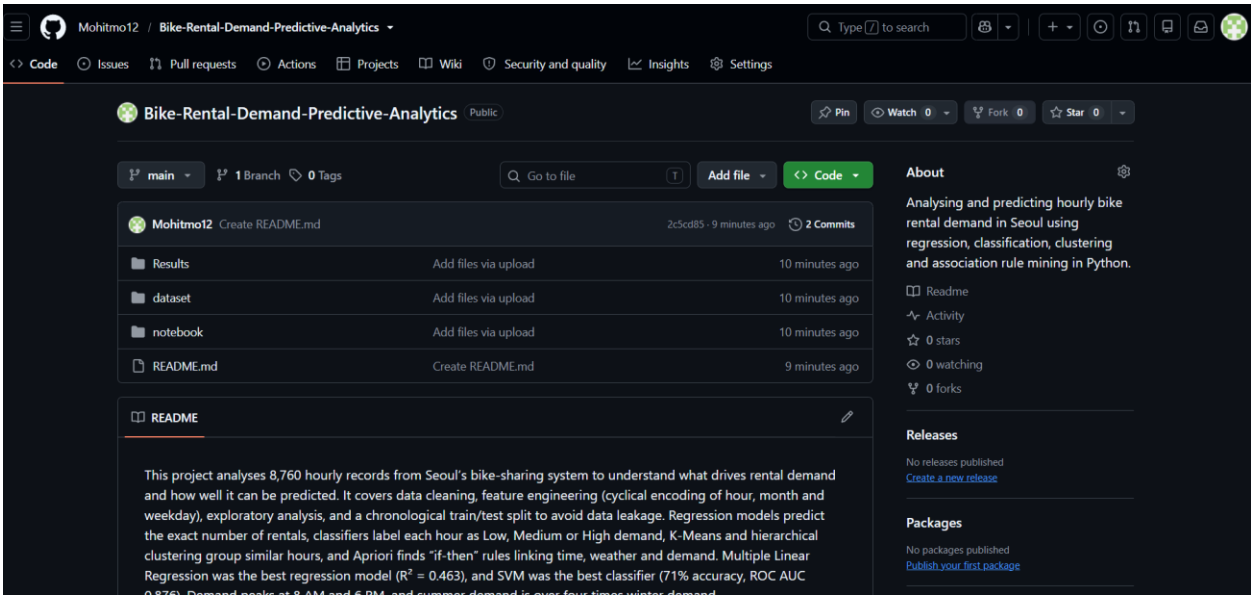
[10] Project source code (GitHub): [add your repository link]

[11] Project summary post (LinkedIn): [add your post link]

Appendix A: GitHub and LinkedIn Screenshots

A.1 GitHub Repository

Link: Seoul Bike Sharing Demand Analysis



A.2 LinkedIn Post

Link: LinkedIn Post

